

MACROFIT AI

Resumen del tercer bloque de trabajo

DIA 3 | Arquitectura por capas, routers, services y schemas

Autor	Matias Gonzalez
Fecha	16 de julio de 2026
Rama de trabajo	feature/backend-setup
Repositorio	github.com/jootuns/macrofit-ai

Objetivo del documento

Registrar el tercer bloque del proyecto, centrado en la reorganizacion del backend en capas, la separacion entre controllers, services y repositories, y la introduccion de schemas y paquetes Python mediante `__init__.py`.

1. Resumen tecnico del dia

Durante este bloque se paso de una API simple con un unico archivo principal a una estructura mas modular y escalable. Se crearon carpetas especificas para api, services, repositories, schemas y models. Se extrajo la ruta /health a un router independiente, se separo la logica en un service y se preparo una estructura de autentificacion con schemas, router y service.

Conceptos consolidados

- main.py actua como punto de entrada y registro de routers.
- api/ contiene controllers o routers que reciben peticiones HTTP.
- services/ contiene la logica de negocio.
- repositories/ contiene el acceso a datos y operaciones CRUD.
- schemas/ contiene modelos de validacion con Pydantic.
- models/ se reserva para modelos de base de datos.
- __init__.py marca las carpetas como paquetes Python e facilita imports limpios.

Arquitectura seguida en este bloque

Cliente -> API/Controller -> Service -> Repository -> Base de datos

La separacion por responsabilidades permitira que el proyecto crezca sin convertir el codigo en un archivo monolitico dificl de mantener.

2. Trabajo realizado

Se crearon las bases de una estructura profesional para el backend y se validaron mediante Swagger los endpoints expuestos por FastAPI.

Estructura creada

Elemento	Estado
backend/app/api	Carpeta para routers/controllers.
backend/app/services	Carpeta para logica de negocio.
backend/app/repositories	Carpeta para acceso a datos.
backend/app/schemas	Carpeta para validacion de datos.
backend/app/models	Carpeta reservada para modelos de base de datos.
__init__.py	Ficheros creados para tratar las carpetas como paquetes Python.

Modulos y archivos creados o repasados

- backend/app/api/health.py -> router /health.
- backend/app/services/health_service.py -> logica de estado de la API.
- backend/app/api/auth.py -> router de autenticacion.
- backend/app/services/auth_service.py -> logica simulada de login y registro.
- backend/app/repositories/user_repository.py -> acceso simulado a usuarios.
- backend/app/schemas/auth.py -> LoginRequest y RegisterRequest.
- backend/app/main.py -> incluye routers y crea la app FastAPI.

Endpoints visibles en Swagger

Metodo	Ruta	Uso
GET	/	Mensaje de bienvenida.
GET	/health	Comprobacion de estado.
POST	/auth/login	Login simulado usando schema.
POST	/auth/register	Registro simulado usando schema.

Pruebas y validaciones

- La API siguio funcionando al mover /health a un router independiente.
- Swagger mostro correctamente los endpoints y los schemas LoginRequest y RegisterRequest.
- Se confirmo que la separacion por capas no rompio el comportamiento de la API.
- Se verifico que los imports entre api, services, schemas y repositories funcionan correctamente.

3. Git y GitHub

El trabajo del dia quedo versionado en la rama de funcionalidad y posteriormente documentado con un PDF especifico del dia 3.

Estado de versionado recomendado

Elemento	Valor
Rama de trabajo	feature/backend-setup
Estado final	working tree clean
Commit esperado para el backend	feat: initialize backend with fastapi
Commit esperado para la documentacion del dia 2	docs: add day 2 project report
Commit recomendado para el dia 3	docs: add day 3 project report

Estado del proyecto al cerrar el dia

- El backend ya no depende de un unico main.py para toda la logica.
- Los modulos de auth y health estan separados por capa.
- Se introdujo la idea de controllers, services, repositories y schemas en un caso real.
- La base del proyecto esta mucho mas escalable y preparada para crecer.

Proximos pasos recomendados

- Crear un modulo de usuarios mas realista para conectar services y repositories con un flujo de negocio completo.
- Empezar a sustituir datos simulados por datos persistidos cuando llegue la fase de base de datos.
- Actualizar el backlog marcando como completadas las tareas del dia 3 relacionadas con arquitectura y modularizacion.
- Guardar este PDF dentro de docs/reports/ y subirlo a GitHub junto con el resto del trabajo del dia.